

Name: Javaughn Stevens

CMSC 204 Project 5 Morse Code Pseudocode (my high-level design)

// Pseudocode for TreeNode.java

Declare a public class `TreeNode<T>`

Declare private variable `data` of type `T`

Declare private variable `leftChild` of type `TreeNode<T>`

Declare private variable `rightChild` of type `TreeNode<T>`

Declare a public constructor `TreeNode` with parameter
`TreeNode<T> node`

If `node` is not null

Set `this.data` equal to `node.data`

Set `this.leftChild` equal to `node.leftChild`

Set `this.rightChild` equal to `node.rightChild`

// end if statement

// end constructor

Declare a public constructor `TreeNode` with parameter `T dataNode`

Set `this.data` equal to `dataNode`

Set `this.leftChild` equal to null

Set `this.rightChild` equal to null

// end constructor

Declare a public `T` method `getData`

return `data`

// end `getData` method

// end of `TreeNode` class and `TreeNode.java` pseudocode

// Pseudocode for MorseCodeTree.java

Declare a public class MorseCodeTree that implements
LinkedConverterTreeInterface<String>

Declare a private TreeNode<String> variable root

Declare a private String variable code

Declare a private String variable letter

Declare public constructor MorseCodeTree

Call buildTree method

// end constructor

Declare public void method addNode with parameters

TreeNode<String> root, String code, String letter

If code length equals 1

If code equals "."

Set root.leftChild equal to new TreeNode<String>(letter)

Else if code equals "-"

Set root.rightChild equal to new TreeNode<String>(letter)

End method (return)

// end of if-else statements

// base case

If code length is greater than 1

If the first character of code equals "."

Call addNode(root.leftChild, substring of code from index 1,
letter) method

Else if first character equals "-"

Call addNode(root.rightChild, substring of code from index 1,
letter)

// end of if-else statement

// recursive case

// end addNode method

```
Declare public void method buildTree  
Set root equal to new TreeNode<String>("")
```

```
// level 1
```

```
Call insert(".", "e") method  
Call insert("-", "t") method
```

```
// level 2
```

```
Call insert("..", "i") method  
Call insert(".-", "a") method  
Call insert("-.", "n") method  
Call insert("--", "m") method
```

```
// level 3 and higher (continue the pattern)
```

```
Call insert("...", "s") method  
Call insert("..-", "u") method  
Call insert("-.-", "r") method  
Call insert("-.-", "w") method  
Call insert("-..", "d") method  
Call insert("-.-", "k") method  
Call insert("---.", "g") method  
Call insert("----", "o") method
```

```
// we continue inserting all alphabetic letters A–Z  
// end buildTree method
```

```
Declare public TreeNode<String> method getRoot  
return root  
// end getRoot method
```

```
Declare public String method fetch with parameter String code  
Return fetchNode(root, code)  
// end fetch method
```

```
Declare public String method fetchNode with parameters
TreeNode<String> root, String code
If code length equals 0
    Return root.data
If first character equals "."
    Return fetchNode(root.leftChild, substring of code from index 1)
Else if first character equals "-"
    Return fetchNode(root.rightChild, substring of code from index 1)
// end of if-else statements
// end fetchNode method
```

```
Declare public void method insert with parameters String code,
String letter
    Call addNode(root, code, letter) method
// end method
```

```
Declare public void method LNRoutputTraversal with parameters
TreeNode<String> root, ArrayList<String> list
If root is not null
    Call LNRoutputTraversal(root.leftChild, list) method
    Add root.data to list
    Call LNRoutputTraversal(root.rightChild, list) method
// end if statement
// end LNRoutputTraversal method
```

```
Declare public void method setRoot with parameter
TreeNode<String> newNode
    Set root equal to newNode
// end setRoot method
```

```
Declare public ArrayList method toArrayList
Declare ArrayList<String> morseCodeList
Call LNRoutputTraversal(root, morseCodeList) method
```

Return morseCodeList
// end toArrayList method

Declare public void method update
Throw a new UnsupportedOperationException
// end update method

// end of MorseCodeTree class and MorseCodeTree.java
pseudocode

// Pseudocode for MorseCodeConverter.java

Declare public class MorseCodeConverter
Declare private String variable code
Declare private File variable codeFile
Declare private static MorseCodeTree variable codeTree initialized
to new MorseCodeTree

Declare a new public constructor MorseCodeConverter
// end of constructor

Declare public static String method printTree
Declare ArrayList<String> variable treeList and set equal to
codeTree.toArrayList()
Declare StringBuilder variable result

For each element in treeList
Append element
Append space
// end of for-each loop
Return result converted back to String
// end printTree method

Declare public static String method convertToEnglish with
parameter String code
Declare StringBuilder variable result

```
Split code using " / " into words array
For each word in words
Split word using " "(space) into letters array
For each letterCode in letters
Append codeTree.fetch(letterCode) to result
// end of nested for-each loop
Append space to result
// end of outer for-each loop
Return trimmed and converted result
// end convertToEnglish method
```

Declare public static String method convertToEnglish with
parameter File codeFile

```
Declare StringBuilder variable result
Open Scanner variable input for codeFile
While input has another line to read
  Read line
  Append convertToEnglish(line) to result
  Append " " (space)
// end of while loop
Close input
Return result converted to string and trimmed
// end convertToEnglish method
```